Inserting Data into a Table

Lecture Notes · Game DB Junior

In this lesson

- How to add data into a table with **INSERT INTO** and **VALUES**
- Why the **order** of columns and values must match
- How to insert **many rows** in one statement

1. The data we want to add

We have our **cities** table. Now we want to add information about four cities: Tokyo, Delhi, Shanghai and Sao Paulo. For each city we have the country, the population and the area. We want to put all these values into the cities table.

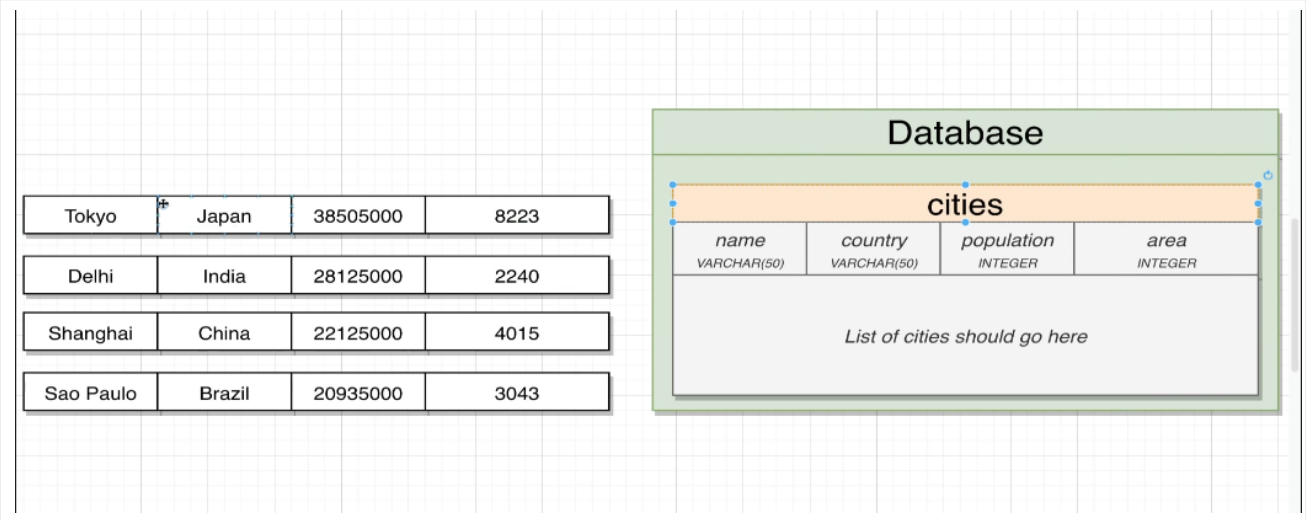


Figure 1 — Four cities that we want to put into the cities table

name	country	population	area
Tokyo	Japan	38505000	8223
Delhi	India	28125000	2240
Shanghai	China	22125000	4015
Sao Paulo	Brazil	20935000	3043

2. The INSERT INTO statement

To add data, we write SQL again. **INSERT INTO** are the keywords we use to insert information into a table.

```
INSERT INTO cities (name, country, population, area)
VALUES ('Tokyo', 'Japan', 38505000, 8223);
```

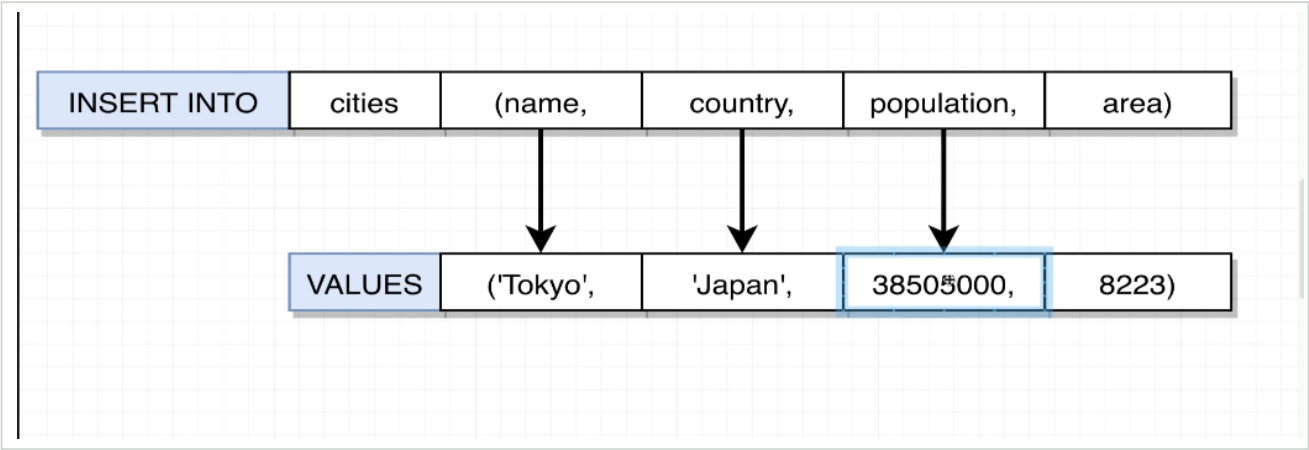


Figure 2 — Each value goes into the column in the same position

Part	What it means
INSERT INTO	Keywords: we want to insert information into a table
cities	The name of the table
(name, country, population, area)	The list of columns we will give values for
VALUES	Keyword: the values come next
('Tokyo', 'Japan', 38505000, 8223)	The values we want to put into the table

- Text values like 'Tokyo' go inside **single quotes** (' '), **not** double quotes.
- When we run it, we see that the insert was successful: **one row** was added to the table.

About the semicolon (;)
The SQL standard says we have to put a semicolon at the end of a statement. In this interface it is not required, so leaving it out is technically OK. But it is highly recommended to add it, especially when you write many statements in one query.

3. The order must match

We listed the columns in this order: name, country, population, area. We could change this order if we wanted. The only rule: the values must be in the **same order** as the columns.

Example of a mistake: we swap **country** and **population** in the column list, but not in the values:

```
INSERT INTO cities (name, population, country, area)
VALUES ('Tokyo', 'Japan', 38505000, 8223);
```

- Now the string 'Japan' would go into the **population** column.
- And 38 million would go into the **country** column.
- This is not what we want. Always give the values in the same order as the column names.

4. Insert many rows in one statement

Writing three separate statements to add three more cities would be tedious. Instead, we can insert **many rows in one statement**:

```
INSERT INTO cities (name, country, population, area)
VALUES
    ('Delhi', 'India', 28125000, 2240),
    ('Shanghai', 'China', 22125000, 4015),
    ('Sao Paulo', 'Brazil', 20935000, 3043);
```

- Each set of parentheses ( ) is **one row**.
- After each set of values, close the parentheses and put a **comma**. Then write the next set of values.
- All rows are inserted in the **same query**. When we run it, we see three rows were inserted.

5. Running the code (my practice)

I wrote one INSERT INTO statement for all four cities, with short comments. I used the numbers from the Wikipedia page today.

```
-- Add new rows to the "cities" table
INSERT INTO cities (
    name,           -- City name
    country,        -- Country name
    population,     -- Number of people
    area            -- Size of the city
)
VALUES
    ('Tokyo', 'Japan', 37785000, 8231),       -- Tokyo city data
    ('Delhi', 'India', 32226000, 2344),       -- Delhi city data
    ('Shanghai', 'China', 24073000, 4333),    -- Shanghai city data
    ('Sao Paulo', 'Brazil', 23086000, 3649);  -- Sao Paulo city data
```

Figure 3 — Inserting four rows in one statement

The Messages tab shows **INSERT 0 4** and "Query returned successfully". The **4** is the number of rows that were added.

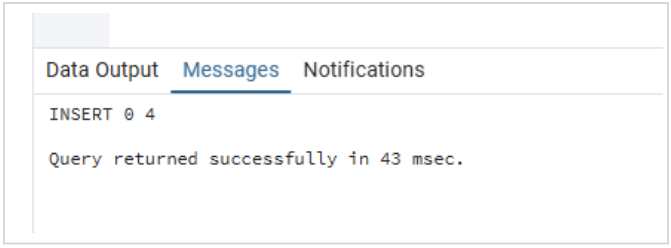


Figure 4 — Four rows were inserted

6. Summary

Idea	Remember
INSERT INTO table (columns)	Choose the table and the columns we give values for
VALUES (...)	The values for one row, inside parentheses
Single quotes	Text values use ' ', not " "
Semicolon ;	Not required here, but highly recommended
Order	Values must be in the same order as the columns
Many rows	Many sets of ( ), separated by commas, in one statement

Next lesson: how to get (retrieve) this information out of the cities table.

Sources
Lecture transcript and lecture images (Figures 1–2)
My own practice screenshots (Figures 3–4)